

ARGUS: Reconciling Conflicting Demands in Unified MLLM Inference through Structure-aware Acceleration

MICRO 2026 Submission #2908 – Confidential Draft – Do NOT Distribute!!

Abstract

Unified Multimodal Large Language Models (MLLMs), which integrate both auto-regressive (AR) and diffusion paradigms into a shared Transformer backbone, have emerged as a key research frontier. Unlike traditional single-paradigm models, unified MLLMs introduce three unique inference challenges that existing accelerators fail to address. First, diverse attention sparsity structures across regions create a fundamental **sparsity-accuracy conflict** that a single exploitation strategy cannot resolve. Second, MLLMs involve diverse conditional prediction workloads that share the same feed-forward network (FFN) weights yet are executed independently, creating **substantial computational redundancy**. Third, the memory-bound AR stage and the compute-bound diffusion stage alternate within a single task, imposing **contradictory resource demands** and resulting in low hardware utilization.

To address these challenges, we propose ARGUS, whose key insight is that each challenge exposes an exploitable structure in MLLM computation that can be captured at the algorithm level and efficiently supported by tailored hardware mechanisms. For the attention operation, a **region-structured sparse attention** applies differentiated sparsity strategies including mixed-precision computation to each attention region. A hybrid sparsity dispatcher overlaps mask generation with computation, achieving 37.06% reduction in attention overhead. For the FFN operation, a **similarity-aware conditional result reuse** bypasses redundant matrix computations for similar tokens. A stateful accumulator unit retains intermediate results on-chip, achieving 39.80% reduction in FFN overhead. Across inference stages, a **stage-adaptive heterogeneous array orchestration** compresses weight bandwidth and balances workloads in the memory- and compute-bound stages, achieving 75.36% hardware utilization. Extensive evaluations demonstrate that ARGUS achieves $1.66\times\sim 1.78\times$ speedup and $1.43\times\sim 18.74\times$ energy efficiency improvement compared to the NVIDIA A100 GPU and state-of-the-art accelerators across various multimodal tasks.

Keywords

Multimodal Large Language Models, Inference Acceleration, Sparse Attention, Mixed-Precision, Hardware-Software Co-design

1 Introduction

Building upon the success of Large Language Models (LLMs), Multimodal Large Language Models (MLLMs) [2, 13, 18, 22] have emerged

as a key research frontier and a pivotal step toward Artificial General Intelligence (AGI) [10, 22]. MLLMs can support both understanding and generating diverse modalities, facilitating broad applications ranging from image editing and generation to world navigation [2]. Recently, unified Transformer structures have demonstrated superior performance across these multimodal tasks [13, 28]. Consequently, in this work, all references to MLLMs specifically denote this auto-regressive (AR) and diffusion fused architecture. However, the inference cost of unified MLLMs remains prohibitively high: compared to traditional understanding-only or generation-only tasks, a single image editing task incurs $2.04\times\sim 5.91\times$ longer latency (BAGEL [2] on GEdit-Bench [12] with identical image resolution), severely limiting their deployment in real-world applications.

The inference challenge of MLLMs is fundamentally different from that of traditional LLMs or diffusion models. As shown in Fig. 1(a) and Fig. 1(b), the AR paradigm is typically memory-bound, while the diffusion paradigm is typically compute-bound. Existing accelerators [3, 6, 9, 11, 14, 19, 27, 29–31] are designed for one or the other, and their optimizations are tailored to the assumptions of a single paradigm. Unified MLLMs, however, force both paradigms to coexist within a shared Transformer backbone, introducing unique computational characteristics at multiple levels: complex attention mechanisms involving multiple region types, conditional prediction¹ across multiple branches, and the coexistence of memory-bound and compute-bound stages. Each characteristic poses new challenges for efficient inference, specifically, we identify three unique challenges (Fig. 1(c)):

Challenge 1: Sparsity-accuracy conflict due to complex attention mechanisms. MLLMs usually use different visual encoders to extract information in both understanding and generation modes. This encoded information manifests as complex distributions and regions in the attention computation, which involves both self-attention and cross-attention. Unlike traditional LLMs or diffusion models that employ a single attention type, unified MLLMs combine cross-attention and self-attention within the same Transformer backbone, producing attention maps with multiple semantically distinct regions that exhibit fundamentally different sparsity structures. Our analysis shows that cross-attention is less sensitive to sparsity (per-token attention scores average only 1.08×10^{-4} , compared to 4.54×10^{-4} for self-attention). Existing sparse attention methods typically apply a uniform strategy across the entire attention map, which struggles to accommodate these structural differences.

Challenge 2: Substantial computational redundancy from diverse prediction workloads. The diffusion paradigm in MLLMs introduces multiple conditional prediction processes that share the same weights yet are executed as independent computation streams.

¹Classifier-Free Guidance [5], an algorithmic innovation that can enhance image generation capabilities by adjusting the degree of adherence to external conditions.

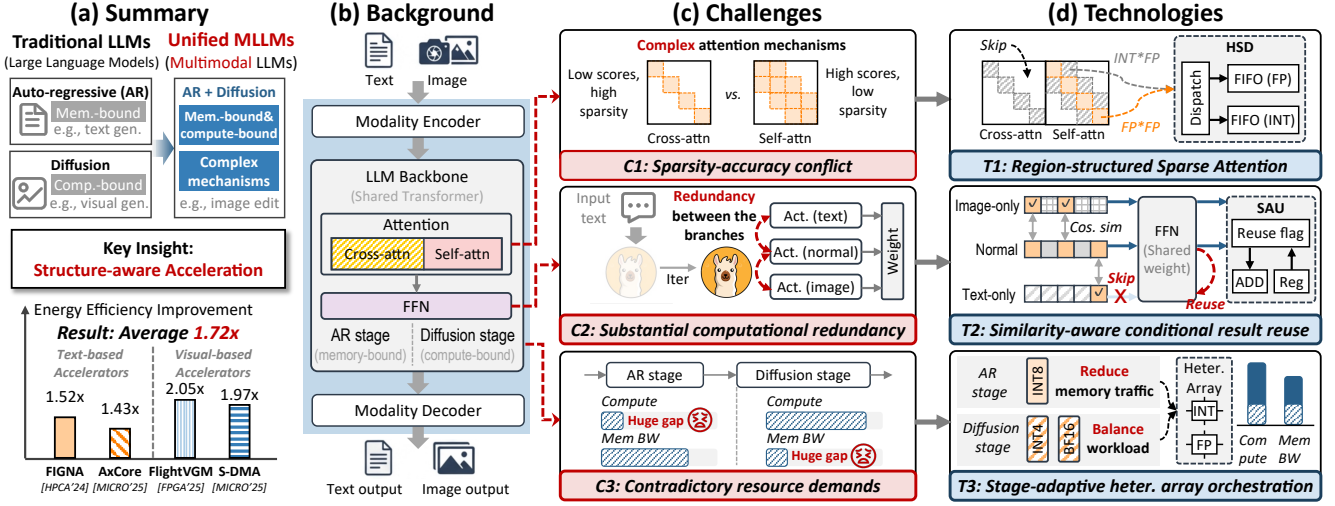


Figure 1: Overview of ARGUS. (a) Summary: unified MLLMs introduce dual bottlenecks; ARGUS achieves consistent energy efficiency gains over single-paradigm accelerators. (b) Background: shared Transformer backbone across AR and Diffusion stages. (c) Three challenges (C1–C3). (d) Three co-designed algorithm-hardware techniques (T1–T3).

Classifier-Free Guidance, for example, triples the feed-forward network (FFN) computation by introducing two additional conditional branches, and FFN operations already account for 78.63% of total computation, making this overhead particularly significant (BAGEL [2] model on GEdit-Bench [12]). However, the degree of similarity varies across tokens and layers, making it difficult to apply a fixed reuse policy and requiring fine-grained runtime decisions on which computations can be safely skipped. Moreover, accumulating results across conditional branches introduces additional complexity: reused and non-reused partial results must be correctly merged, demanding careful hardware orchestration to maintain correctness while exploiting redundancy.

Challenge 3: Contradictory resource demands across inference stages. MLLMs alternate between two different inference paradigms within a single task. The AR stage (e.g., text generation) is memory-bound, where the bottleneck lies in fetching large FFN weight matrices from off-chip memory. The diffusion stage (e.g., image generation) is compute-bound, where the bottleneck lies in saturating the compute array with sufficient arithmetic intensity. These two stages impose contradictory demands on the same hardware: designs optimized for one stage inevitably underutilize resources in the other, resulting in an average GPU utilization of only 21.95% on image editing tasks (Same configuration as above).

Existing accelerators cannot effectively address these challenges. Text-based accelerators [6, 14, 27, 29, 30] mainly target memory-bound AR workloads through either weight quantization or uniform attention sparsifying, but they cannot handle heterogeneous attention characteristics or the compute-bound diffusion stage. Visual-based accelerators [3, 9, 11, 19, 31] exploit sparsity or similarity in diffusion models, but they are designed for single-paradigm workloads and fail to capture region-specific attention patterns or conditional redundancy in unified MLLMs. The fundamental

limitation shared by prior work is the assumption of a fixed computational paradigm, whereas unified MLLMs demand simultaneous handling of all three challenges.

To address these challenges, we propose ARGUS, whose key insight is that each challenge exposes an exploitable structure in MLLM computation that can be captured at the algorithm level and efficiently supported by tailored hardware mechanisms built upon a shared heterogeneous compute substrate. As illustrated in Fig. 1(d), the main contributions are as follows:

- A **region-structured sparse attention** that applies differentiated sparsity strategies to each attention region, including block-skipping for cross-attention and mixed-precision computation for self-attention. A *hybrid sparsity dispatcher (HSD)* leverages cross-region structural similarity to overlap mask generation with computation, achieving 37.06% reduction in attention overhead.
- A **similarity-aware conditional result reuse** that detects activation similarity across conditional branches and bypasses redundant FFN computations for similar tokens. A *stateful accumulator unit (SAU)* retains intermediate results on-chip for efficient reuse across branches, achieving 39.80% reduction in FFN overhead.
- A **stage-adaptive heterogeneous array orchestration** that dynamically adapts weight representation and mapping strategy to the current inference stage, compressing weight bandwidth in the memory-bound AR stage and balancing mixed-precision workloads across heterogeneous arrays in the compute-bound diffusion stage, achieving 75.36% hardware utilization.

Extensive evaluations across image editing, generation, and understanding tasks demonstrate that ARGUS achieves $1.66\times\sim 1.78\times$ speedup and $1.43\times\sim 18.74\times$ energy efficiency improvement over the NVIDIA A100 GPU and state-of-the-art accelerators.

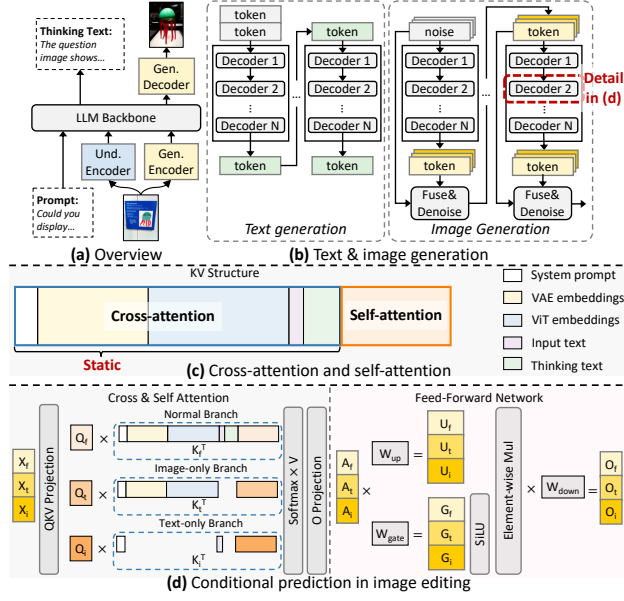


Figure 2: (a) The unified MLLM framework, exemplified by BAGEL [2]. (b) Text generation uses autoregressive decoding; image generation uses diffusion-based denoising. (c) KV cache partitioned into a static cross-attention region and a dynamic self-attention region. (d) Conditional prediction executes three branches over shared attention and FFN weights.

2 Background and Motivation

2.1 Multimodal Large Language Models

Recent MLLMs are designed to understand diverse forms of input and generate outputs in one or more modalities in a unified manner [28]. Additionally, to enhance the performance of these models, several works [2] introduce Chain of Thought (CoT) [24] into MLLMs, which results in interleaved thinking text and other modality content generated in one turn, pushing the system toward further complexity.

A commonly used framework for these models can be separated into three parts: modality encoder, LLM backbone, and modality decoder. Fig. 2 illustrates the framework for MLLMs, which takes BAGEL [2], one of the SOTA open source MLLMs, as an example. Fig. 2(a) depicts a typical multimodal task, image editing, which is chosen for its inherent characteristics of both understanding and generation. Although additional encoders and decoders are added into the system, the bottleneck of the system is still the LLM backbone, as the parameters between these encoders, decoders, and the LLM backbone are usually 1–2 orders of magnitude apart.

2.2 Transformer and Attention

The LLM backbone in MLLMs shares a similar structure with traditional LLMs, both of which leverage Transformer and attention as their skeleton. However, the actual dataflow diverges for diverse task types in MLLMs. As depicted in Fig. 2(b), MLLMs generate text outputs in an autoregressive manner with causal attention. In

contrast, they switch to a diffusion-based manner to predict noise or velocity using bidirectional attention in image generation.

Fig. 2(c) visualizes the cross-attention and self-attention regions for Keys and Values. Here, cross-attention is used to handle multi-modal inputs and contains five different regions at most, which are the system/input prompt, VAE/ViT image embeddings, and thinking text, respectively. While decoding, the cross-attention region is static. Furthermore, generation tasks also comprise conditional prediction [5] to improve the quality of generated content. Fig. 2(d) exhibits three branches for conditional prediction in image editing tasks, which will triple the computation compared to the naive Transformer. The normal branch leverages the full KV cache in cross-attention. The image-only branch alters the system prompts and the KV cache that is only associated with images. Similarly, the text-only branch will alter the system prompts and the text contents in the KV cache.

2.3 Motivation

Unified MLLMs introduce three unique computational characteristics that existing accelerators fail to address. Text-based accelerators [6, 30] are designed for single-paradigm LLM workloads and apply uniform compression strategies across all operations. Visual-based accelerators [11, 31] target compute-bound generative vision models but cannot handle the memory-bound AR stage. Neither class of accelerator is equipped to handle the heterogeneous demands of unified MLLMs. In this subsection, we provide detailed profiling analysis to substantiate each challenge.

2.3.1 Diverse Sparsity Structures across Attention Regions. MLLMs employ both cross-attention and self-attention within the same Transformer backbone, yet the two exhibit distinctly different sparsity characteristics. As shown in Fig. 3(a) (left), we visualize the attention score distribution across different regions on the BAGEL model. Cross-attention spans multiple sub-regions (VAE, ViT, and Text embeddings) and spreads its scores broadly, with a low per-token average of 1.08×10^{-4} . Self-attention concentrates its scores on a small set of critical tokens near the diagonal, with a significantly higher per-token average of 4.54×10^{-4} .

This distributional difference leads to drastically different tolerance to sparsity. As shown in Fig. 3(a) (right), we measure the accuracy (GEdit-Bench) of cross-attention and self-attention under increasing sparsity ratios. Cross-attention tolerates sparsity up to 90% with only marginal accuracy degradation (from 6.355 to 6.233, a drop of merely 0.122), indicating that the vast majority of its computation is redundant and can be safely skipped. In contrast, self-attention is highly sensitive to sparsity: at the same 90% sparsity ratio, accuracy collapses from 6.355 to 0.498, a catastrophic drop of 5.723. These results demonstrate that a uniform sparsity strategy cannot simultaneously serve both attention types: aggressive skipping that benefits cross-attention would destroy self-attention accuracy, while a conservative strategy that protects self-attention would leave substantial cross-attention redundancy unexploited.

2.3.2 High Similarity across Conditional Branches. The diffusion paradigm in MLLMs introduces conditional prediction (Classifier-Free Guidance, CFG [5]), which triples the FFN computation by executing three branches (normal, image-only, and text-only branches)

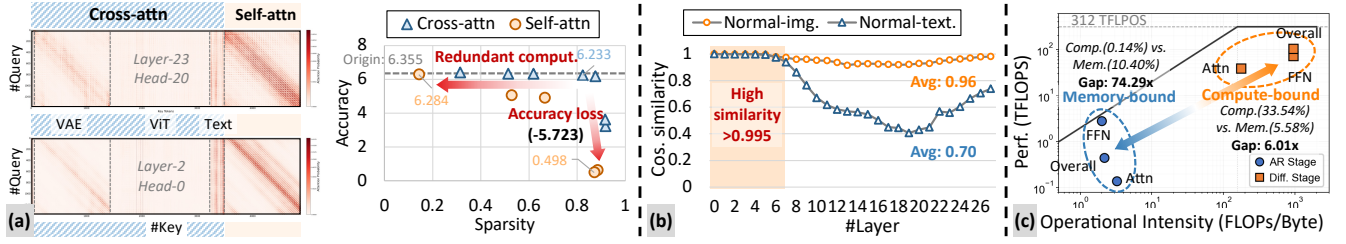


Figure 3: (a) Attention score distribution (left) and accuracy under increasing sparsity ratios (right) for cross-attn and self-attn. (b) Layer-wise cosine similarity between the normal branch and each conditional branch. (c) Roofline model showing AR-stage operators are memory-bound and Diffusion-stage operators are compute-bound.

that share the same weights. Although these branches operate under different conditions, we find that their intermediate activations exhibit high similarity.

As shown in Fig. 3(b), we measure the layer-wise cosine similarity between the normal branch and the other two conditional branches across all layers in the BAGEL model. The normal-image branch maintains consistently high similarity throughout all layers, with an average of 0.96, while the normal-text branch exhibits a declining trend in later layers but still averages 0.70. This high similarity arises from two factors: first, residual connections in the Transformer architecture dominate the activation values, causing outputs to remain close across branches. Second, all branches share the same FFN weights, further limiting divergence. This similarity-induced redundancy is substantial yet predictable. For tokens whose activations are sufficiently similar across branches, their FFN outputs will also be nearly identical, making it possible to reuse results from the normal branch and skip redundant matrix computations.

2.3.3 Alternating Memory-bound and Compute-bound Stages. Unified MLLMs alternate between two different inference paradigms within a single task. The AR stage (e.g., text generation) processes one token at a time, making it memory-bound. The diffusion stage (e.g., image generation) processes many tokens simultaneously, making it compute-bound.

As shown in Fig. 3(c), we profile the hardware utilization of an NVIDIA A100 GPU on the image editing task using the BAGEL model on GEdit-Bench [12]. The overall compute utilization is only 21.95%, revealing a severe mismatch between the two stages. In the AR stage, FFN weight loading dominates execution, resulting in high memory bandwidth pressure but near-zero compute utilization. The gap between memory and compute utilization reaches 74.29 $\times$. In the Diffusion stage, batched token processing shifts the bottleneck toward computation, yet the gap remains 6.01 $\times$ due to the overhead of conditional branches. As confirmed by the roofline analysis in Fig. 3(c), AR-stage operators cluster in the memory-bound region while Diffusion-stage operators fall in the compute-bound region, with no single hardware configuration capable of simultaneously bridging both gaps. This low overall utilization stems from the fundamental mismatch between the two stages.

3 Overview of ARGUS

Fig. 4 illustrates the overall architecture of ARGUS. At its core sits the **Heterogeneous Computing Core (HCC)**, which comprises

an INT-FP computing array for energy-efficient low-precision computation [6] and an FP-FP computing array for high-precision BF16 MAC operations. These two arrays serve as the unified compute backbone shared across all three optimization techniques. The **Configurable Reduction Unit (CRU)** complements the HCC by handling non-matrix operations. As shown in the expanded view in Fig. 4, the CRU consists of a reduction tree supporting sum, max, and min operations, a post-reduction unit for sqrt, reciprocal, and exp computations, a scale calculator for group-wise quantization parameter derivation, lookup tables for sigmoid and exp activations, and 32 parallel element-wise lanes for operations such as multiplication, addition, and activation functions. This makes the CRU responsible for tasks including on-chip INT8-to-BF16 de-quantization, inter-token similarity calculation, and non-linear operations such as soft-max, RMSNorm, and SiLU that appear throughout the model. The **Scheduler** manages the overall dataflow and implements the stage-adaptive orchestration strategy (T3). As shown in its expanded view, it maintains stage registers, a stage controller that selects weight formats and array mapping strategies at each AR-to-Diffusion stage transition, and an address generator that translates logical tile indices to physical off-chip memory addresses. The **Unified Buffer (UB)** provides on-chip storage for intermediate activations, while the Left and Right Buffers serve as staging areas for operands flowing into the HCC. The **Hybrid Sparsity Dispatcher (HSD)** and the **Stateful Accumulator Unit (SAU)** are dedicated hardware units introduced to support T1 and T2, respectively. Their designs are detailed in Sections 4.2 and 5.2.

The key insight behind ARGUS is that each of the three challenges identified in Section 2.3 exposes an exploitable structure in MLLM computation, which can be captured at the algorithm level and efficiently supported by tailored mechanisms built upon this shared substrate. Specifically, ARGUS introduces three co-designed techniques:

Region-structured sparse attention (Section 4) addresses the sparsity-accuracy conflict (C1). At the algorithm level, it applies differentiated sparsity strategies to each attention region: block-skipping for cross-attention and mixed-precision computation for self-attention. At the hardware level, the HSD overlaps heterogeneous mask dispatching with computation by exploiting cross-region similarity, enabling efficient utilization of both arrays for attention workloads.

Similarity-aware conditional result reuse (Section 5) addresses the substantial computational redundancy challenge (C2).

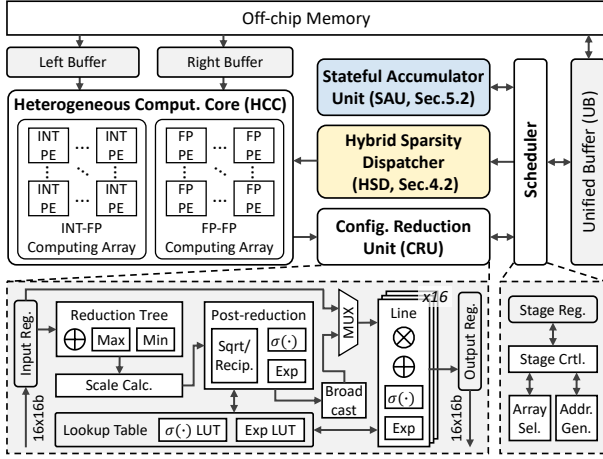


Figure 4: Overall architecture of ARGUS.

At the algorithm level, it detects activation similarity across conditional branches and bypasses redundant FFN matrix computations for similar tokens. At the hardware level, the SAU retains intermediate conditional results on-chip and performs efficient accumulation through a reuse-maximizing out-of-order execution strategy, avoiding repeated memory accesses.

Stage-adaptive heterogeneous array orchestration (Section 6) addresses the contradictory resource demands challenge (C3). Rather than introducing dedicated hardware, this technique is implemented entirely through the Scheduler’s stage-aware dispatching policy. In the memory-bound AR stage, it compresses FFN weight bandwidth by loading weights in INT8 and de-quantizing on-chip via the CRU before computation. In the compute-bound diffusion stage, it balances workload across the heterogeneous arrays by distributing weights in INT4 and BF16 formats to the INT-FP and FP-FP arrays respectively, maximizing the utilization.

4 Region-structured Sparse Attention

This section presents our first algorithm-hardware co-designed technique that addresses Challenge 1: sparsity-accuracy conflict due to complex attention mechanisms. We first describe the algorithm that applies differentiated sparsity strategies to each attention region based on its structural characteristics (Section 4.1), and then present the Hybrid Sparsity Dispatcher (HSD) that efficiently maps the resulting region-structured workload onto hardware (Section 4.2).

4.1 Algorithm Design

As established in Section 2.3, cross-attention and self-attention exhibit fundamentally different sparsity structures that demand differentiated treatment. Cross-attention, which handles multimodal understanding, spreads its scores broadly across many tokens, exhibiting high sparsity that tolerates aggressive block-skipping. Self-attention, which focuses on generation, concentrates its scores on a small set of critical tokens, forming a structured but fragile sparsity pattern where aggressive skipping discards essential information. Building on these observations, we propose a **region-structured sparse attention** that applies differentiated sparsity strategies tailored to the structural characteristics of each attention region.

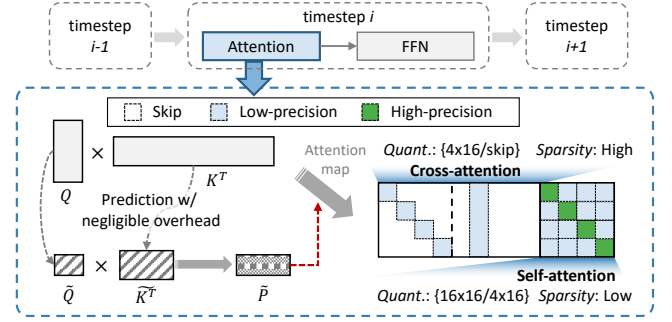


Figure 5: Region-structured sparse attention.

The overall workflow is illustrated in Fig. 5. The method operates in two phases: mask prediction and region-aware computation.

Phase 1: Mask prediction. We first extract compressed representations $\tilde{Q}$ and $\tilde{K}^T$ from the original Q and K^T matrices using grouped average pooling along the token dimension. Through the careful design of the group size G ($G=16$ in our design), we achieve high prediction accuracy while incurring only negligible additional overhead. This is because the prediction complexity is $O((\frac{N}{G})^2 D)$, which is significantly smaller than the $O(N^2 D)$ complexity of the full attention computation, where N and D denote the number of tokens and the model’s hidden dimension, respectively. The compressed representations are multiplied to produce approximate attention scores, from which binary attention masks $\tilde{P}$ are derived by thresholding.

Phase 2: Region-aware computation. Based on the predicted attention masks $\tilde{P}$, we apply different computation strategies to cross-attention and self-attention regions: **Cross-attention regions:** We apply an aggressive sparsity strategy: unmasked regions are computed at low precision (INT4×BF16) and masked regions are skipped entirely. This is sound because cross-attention exhibits high, broadly distributed sparsity, and skipped blocks carry negligible score mass. **Self-attention regions:** We apply a conservative sparsity strategy: unmasked regions are computed at full precision (BF16×BF16), and masked regions are retained but computed at reduced precision (INT4×BF16) rather than skipped. This is necessary because self-attention sparsity is structurally concentrated: the masked regions still contribute meaningful scores, and discarding them entirely would cause noticeable accuracy degradation. By retaining masked regions at lower cost, we preserve essential information while still reducing computation.

4.2 Hybrid Sparsity Dispatcher

While the region-structured sparse attention algorithm reduces the total computation, efficiently mapping the resulting workload onto the HCC presents a significant challenge. The algorithm produces two types of computation with different spatial distributions across the attention map, and these must be dispatched to the appropriate computing arrays without introducing pipeline stalls.

Our key idea is to **overlap the dispatching process with computation by exploiting cross-region similarity**. As shown in Fig. 6, we observe a high degree of similarity between the attention masks of the VAE region in cross-attention and those of self-attention. Based on this observation, we can directly transfer the

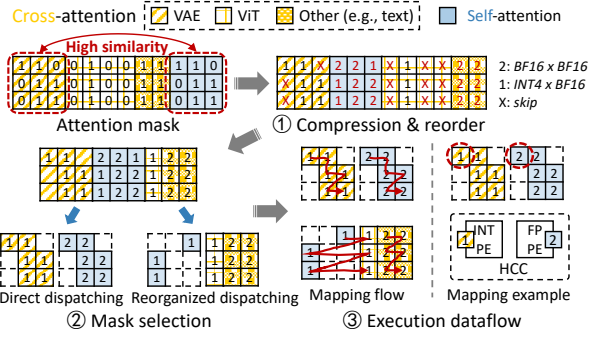


Figure 6: Cross-region similarity between VAE region masks in cross-attention and self-attention masks, which enables direct mask reuse and overlapped dispatching.

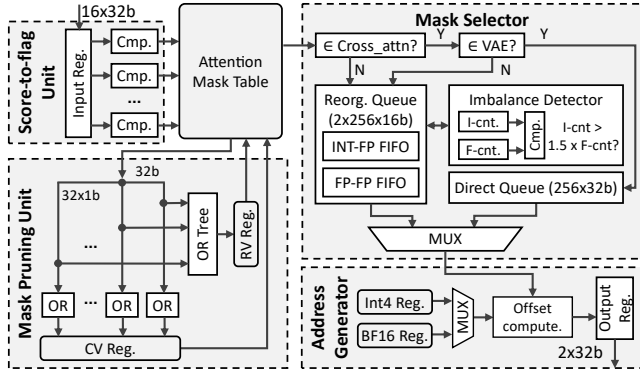


Figure 7: Hardware architecture of the Hybrid Sparsity Dispatcher (HSD), consisting of the score-to-flag unit, mask pruning unit, mask selector, and address generator.

attention masks from the former to the latter with almost no impact on results, and propose a *direct-reorganized attention mask selection* strategy that divides dispatching into two concurrent streams. In the **direct stream**, the VAE region in cross-attention and the unmasked parts of self-attention are dispatched simultaneously: they require different computation types (low-precision and high-precision, respectively) and can be sent directly to the corresponding arrays. In the **reorganized stream**, the remaining parts (including the non-VAE regions of cross-attention and the masked portions of self-attention) are reorganized and merged by computation type before being sent to the HCC. This reorganization is overlapped with the direct stream computation, avoiding additional latency.

Fig. 7 shows the hardware architecture of the HSD, which consists of four main components. The **score-to-flag unit** receives the approximate attention scores from the HCC and converts them into binary attention masks through a parallel comparator array, storing the results in an attention mask table for downstream use. The **mask pruning unit** then performs coarse-grained sparsity pruning: an OR tree reduces each row and column of the mask to a single bit, and any row or column whose reduced value is zero is identified as fully sparse and eliminated before fine-grained dispatching, avoiding unnecessary computation. The **mask selector** implements the direct-reorganized attention mask selection strategy using a two-level decision: tiles belonging to the VAE region

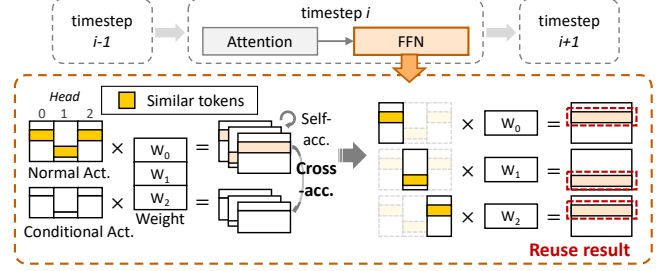


Figure 8: Similarity-aware conditional result reuse.

of cross-attention are sent directly to the direct queue, while all remaining tiles are classified by computation type and routed into separate INT-FP and FP-FP FIFOs within the reorganized queue. An imbalance detector monitors the relative lengths of the two FIFOs and, upon detecting that INT-FP demand significantly exceeds FP-FP demand, redirects a portion of the INT-FP tasks to the FP-FP array to restore balance. Finally, the **address generator** maintains separate register paths for INT4 and BF16 operands, and computes the physical buffer addresses for each dispatched tile through an offset computation unit, producing dual output addresses to feed both computing arrays simultaneously.

5 Similarity-aware Conditional Result Reuse

This section presents our second algorithm-hardware co-designed technique that addresses Challenge 2: substantial computational redundancy from diverse prediction workloads. We first describe the algorithm that detects activation similarity across conditional branches and bypasses redundant FFN computations (Section 5.1), and then present the Stateful Accumulator Unit (SAU) that efficiently supports on-chip result reuse (Section 5.2).

5.1 Algorithm Design

As established in Section 2.3, the conditional prediction mechanism in MLLMs triples the FFN computation by executing three branches that share the same weights. This overhead is particularly severe because the intermediate dimension of FFNs is very high (typically $2.75\sim 5.29\times$ that of the hidden dimension [2, 13]), making FFNs a dominant contributor to the overall inference cost. However, our profiling reveals that activations across conditional branches exhibit high similarity, creating substantial redundancy that can be exploited. Building on this observation, we propose a **similarity-aware conditional result reuse** method that measures activation similarity and skips redundant matrix computations for similar tokens. The overall workflow is illustrated in Fig. 8.

Step 1: Similarity detection. Taking activation values under the normal branch as references, we compute the cosine similarity between identically indexed tokens across different conditional branches (image-only and text-only). The similarity detection overhead is negligible: the prediction complexity is $O(ND)$, whereas the FFN computation complexity is $O(NDD_i)$, where D_i is the intermediate dimension. D_i is typically in the thousands, with values of 5.6k and 18.9k in [13] and [2], respectively.

Step 2: Multi-head grouping. We employ a multi-head grouping approach to fully explore the similarity between the conditional

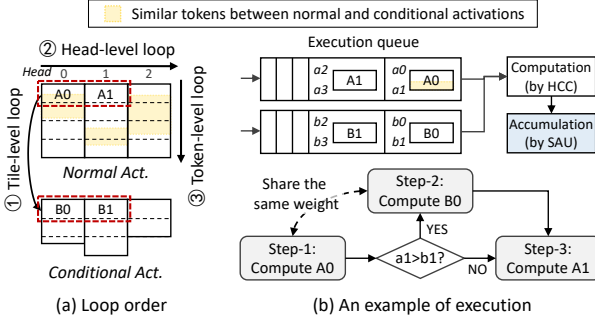


Figure 9: Reuse-maximizing out-of-order execution strategy. (a) Three levels of loop priority. (b) Example of out-of-order execution based on index relationships.

and normal activations at a finer granularity. Rather than comparing entire activation vectors, we divide them into multiple groups and evaluate similarity within each group. This approach captures partial similarity that would be missed by a single global comparison, enabling more aggressive reuse.

Step 3: Selective computation and reuse. For tokens whose activations are sufficiently similar across branches, we skip the matrix computation in the conditional branch and directly reuse the result from the normal branch by accumulating it onto the normal process result. Due to the multi-head grouping, some partial results also require self-accumulation within the same branch.

5.2 Stateful Accumulator Unit

While the similarity-aware conditional result reuse algorithm reduces the total FFN computation, efficiently handling the accumulation of intermediate results on hardware presents a significant challenge. The similarity distribution varies across different heads and layers, meaning the set of tokens that can be reused changes dynamically. A naive implementation would require repeated loading of intermediate activations from off-chip memory for accumulation, negating much of the computational savings.

Our key idea is to **keep reusable intermediate data and the remaining results from the conditional process together on-chip as much as possible**, realized through a *reuse-maximizing out-of-order execution* strategy. As shown in Fig. 9(a), we organize the computation into three levels of loop priority: tile-level, head-level, and token-level, where different activations share the same weights at the tile level to avoid redundant weight loading. However, because the positions of similar tokens differ across heads, this loop order would naively cause repeated loading of conditional activations. To address this, as illustrated in Fig. 9(b), we reorder the computation sequence so that tiles requiring accumulation from the normal branch are computed in close temporal proximity to the corresponding conditional tiles, ensuring intermediate results remain on-chip and can be directly accumulated without off-chip memory round-trips.

Fig. 10 shows the hardware architecture of the SAU, which is positioned between the HCC output and the result buffer. Each incoming tile result is accompanied by a reuse flag that encodes the accumulation mode for that tile: whether it should be accumulated with the normal branch result, the image-only branch

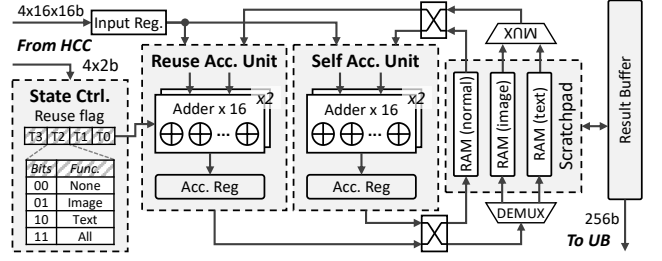


Figure 10: Hardware architecture of the Stateful Accumulator Unit (SAU), consisting of the state controller, reuse accumulation unit, self accumulation unit, and on-chip RAM banks.

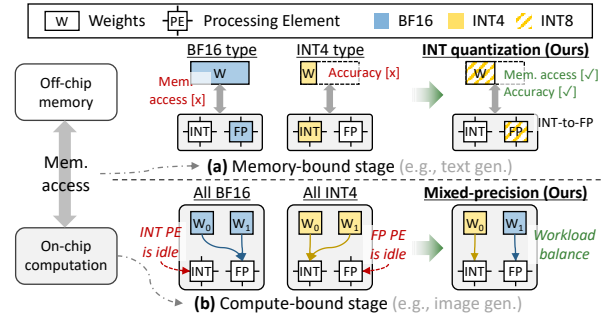


Figure 11: Stage-adaptive heterogeneous array orchestration.

result, the text-only branch result, or all branches. The **state controller** decodes this flag and drives the downstream routing and accumulation logic accordingly. The **reuse accumulation unit** handles cross-branch accumulation: when a conditional branch tile is similar to its normal branch counterpart, the unit reads the corresponding normal branch result from RAM and accumulates it onto the incoming tile, avoiding redundant matrix computation. The **self accumulation unit** handles within-branch partial sum accumulation arising from multi-head grouping, where partial results within the same branch must be summed before the final output. Three separate on-chip RAM banks store intermediate results for the normal, image-only, and text-only branches respectively, while a scratchpad holds partial sums during accumulation. A MUX and DEMUX pair controls the read and write paths into these banks under the direction of the state controller. The fully accumulated tile result is then sent to the result buffer for writeback to the unified buffer.

6 Stage-adaptive Heterogeneous Array Orchestration

This section presents our third technique that addresses Challenge 3: contradictory resource demands across inference stages. Unlike the previous two techniques that introduce dedicated hardware units, this technique leverages the existing heterogeneous computing arrays and dynamically adapts the weight representation and mapping strategy to the current inference stage. We describe the optimization for the memory-bound AR stage (Section 6.1) and the compute-bound diffusion stage (Section 6.2) separately.

6.1 Memory-bound Stage Optimization

In the AR stage (*e.g.*, text generation), the model generates one token at a time, and the bottleneck lies in fetching large FFN weight matrices from off-chip memory. The weight data volume of the FFNs dominates the entire auto-regressive decoding process, owing to the large intermediate dimensions within FFNs.

Fig. 11(a) illustrates the optimization for this stage. The core idea is to reduce the volume of FFN weights that must be fetched from off-chip memory, even at the cost of a moderate increase in on-chip computation. Given the numerical sensitivity of text generation, we maintain all activations in the BF16 format. However, using the natively supported BF16 or INT4 data types for FFN weights poses a dilemma: BF16 incurs significant memory bandwidth overhead, while INT4 results in noticeable degradation in model accuracy.

To resolve this dilemma, we load the FFN weights in INT8 format from off-chip memory and de-quantize them back to BF16 on-chip using the CRU before computation. The de-quantized BF16 weights are then sent to the FP-FP computing array for high-precision FFN computation. This approach reduces the memory traffic for FFN weights in text generation by 46.88% compared to BF16 storage, with negligible impact on model accuracy.

6.2 Compute-bound Stage Optimization

In the diffusion stage (*e.g.*, image generation), the model processes many tokens simultaneously, and the bottleneck shifts from memory bandwidth to compute throughput. Although the algorithmic optimizations in Sections 4 and 5 already reduce the FFN computational workload, relying on a single data type for FFN weights still limits the ability to fully utilize the available compute power of the heterogeneous arrays.

Fig. 11(b) illustrates the optimization for this stage. The core idea is to employ mixed-precision FFN weights to balance the workload across the heterogeneous INT-FP and FP-FP computing arrays, mitigating the workload imbalance inherent in single-precision execution. Specifically, we store FFN weights in two formats: a portion in INT4 for the INT-FP array and the remainder in BF16 for the FP-FP array. This enables both arrays to operate concurrently on the same FFN computation, significantly improving overall hardware utilization.

Although text and image generation share the same FFN weights, we can store copies in different numerical formats in off-chip memory. This is feasible because the parameters of current MLLMs are modest enough to be accommodated by typical off-chip memory capacities. Furthermore, this mixed-precision design directly reuses the existing heterogeneous hardware architecture without requiring any additional design modifications.

6.3 Stage Switching Mechanism

The stage-adaptive orchestration switches between the two weight representation strategies depending on the current inference stage. The stage type (AR or Diffusion) is determined by the model’s inference schedule and is known prior to execution. Specifically, the FFN weights are pre-stored in off-chip memory in two formats: INT8 for the AR stage and mixed INT4+BF16 for the Diffusion stage. At each stage transition, the controller selects the appropriate weight format and mapping strategy. This switching incurs negligible overhead,

as it only involves changing the memory access pattern and does not require any runtime computation or data conversion between stages. The de-quantization from INT8 to BF16 in the AR stage is performed on-chip by the CRU as weights are loaded, overlapping with computation.

7 Evaluation

7.1 Experiment Setup

7.1.1 Models. We evaluate ARGUS on two state-of-the-art open-source unified MLLMs: JanusFlow-1.3B [13] (DeepSeek) and BAGEL-7B-MoT [2] (ByteDance), both of which follow an autoregressive and diffusion interleaved generation paradigm. These two models differ in scale (1.3B vs. 7B) and attention mechanism: JanusFlow generates images based on a causal attention mask, whereas BAGEL uses a bidirectional attention mask to capture correlations between image tokens.

7.1.2 Tasks and Benchmarks. We evaluate our method on three types of tasks, each corresponding to a specific benchmark: (1) For image editing, we use GEdit-Bench [12] to evaluate editing quality in a real-world scenario. GEdit-Bench generates an evaluation score through a SOTA Vision-Language Model; in our experiment, we select Qwen2.5-VL-72B-Instruct for evaluation. Note that JanusFlow does not support image editing tasks, so GEdit-Bench results are only reported for BAGEL. (2) For image generation, we calculate the overall score in GenEval [4]. (3) For image understanding, we select MM-Vet [26] to provide a robust evaluation. These three tasks collectively cover the full spectrum of MLLM workloads, ranging from AR-dominated (image understanding) through mixed AR+Diffusion (image editing) to Diffusion-dominated (image generation), enabling a comprehensive assessment of ARGUS’s adaptability.

7.1.3 Baselines. We select five representative baselines spanning general-purpose hardware and domain-specific accelerators: (1) **NVIDIA A100 GPU**, as the general-purpose baseline. GPU results are obtained from real hardware measurements using the official model implementations with Flash Attention [1]. GPU power consumption is measured using NVIDIA’s System Management Interface (*nvidia-smi*) [15], yielding an average of 380.4 W during inference. (2) **FIGNA** [6] and **AxCore** [30], as text-based accelerators that design mixed-precision computational units for LLMs supporting low-bit quantized weights (INT4/8 and FP4 formats). Since their low-precision computing arrays cannot meet the high-precision demands of MLLMs, we augment them with additional BF16 computing arrays for fair comparison. (3) **FlightVGM** [11] and **S-DMA** [31], as visual-based accelerators that exploit activation sparsity and similarity in generative vision models.

7.1.4 Evaluation Methodology. We implement the proposed hardware modules in SystemVerilog at the Register-Transfer Level (RTL) and synthesize them using Synopsys Design Compiler [21] under a 28nm CMOS process. For the HCC, we adapt the evaluation results from FIGNA [6], normalizing the process technology to 28nm CMOS based on the scaling methodology from [17, 20]. Table 3 shows the area and power breakdown of ARGUS.

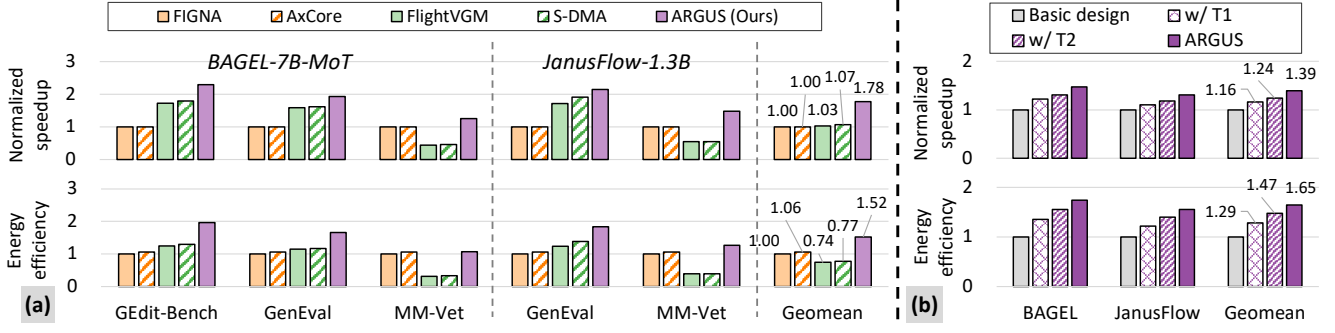


Figure 12: (a) The comparison of normalized speedup and energy efficiency. (b) Performance breakdown.

To evaluate performance, we develop an architectural simulator for ARGUS based on SCALE-Sim v3 [16] and Ramulator [8]. For a fair comparison among domain-specific accelerators, we align the clock frequency (500 MHz), the number of MAC units (2048), and the off-chip memory bandwidth (32 GB/s) across all accelerator designs, following the common practice in prior studies [6, 11, 30, 31]. Since ARGUS and the DSA baselines operate at a much smaller compute budget than the A100 GPU, we conduct two separate comparisons: Section 7.2 compares ARGUS against DSA baselines under a unified hardware configuration (aligned clock frequency, MAC count, and memory bandwidth). Section 7.4 compares ARGUS against the A100 GPU directly, using an analytical scale-out model and single-tile energy efficiency analysis calibrated against simulator measurements.

7.2 Overall Performance

Fig. 12(a) shows the normalized speedup and energy efficiency improvements of different methods across multiple models and benchmarks. We conduct the comparison under the condition of comparable model accuracy for all methods to ensure fairness. Across all DSA baselines, ARGUS achieves a geomean speedup of 1.78 $\times$ and energy efficiency improvement of 1.52 $\times$. This is because ARGUS, through its algorithm-hardware co-design, effectively eliminates redundant computation and achieves efficient hardware execution.

From a speedup perspective, ARGUS achieves 1.78 $\times$, 1.78 $\times$, 1.73 $\times$, and 1.66 $\times$ speedup over FIGNA, AxCore, FlightVGM, and S-DMA, respectively. FIGNA and AxCore exhibit a performance gap compared to ARGUS because they cannot simultaneously leverage the heterogeneous computing arrays. FlightVGM and S-DMA employ a unified computing array, and their single compression method can only remove limited computational redundancy.

From an energy efficiency perspective, ARGUS achieves 1.52 $\times$, 1.43 $\times$, 2.05 $\times$, and 1.97 $\times$ improvement in energy efficiency over FIGNA, AxCore, FlightVGM, and S-DMA, respectively. S-DMA and FlightVGM suffer from low energy efficiency due to high hardware overhead. Similarly, the gains from FIGNA and AxCore are severely limited because their low-precision computing arrays cannot exploit sparsity. In contrast, ARGUS fully utilizes heterogeneous computing arrays, achieving superior speedups and energy efficiency through algorithm-hardware co-design.

Analysis of cross-benchmark variation. The variation in performance across benchmarks reflects the inherent diversity of MLLM workloads rather than instability of ARGUS. These tasks follow distinct inference paradigms and impose different bottlenecks

(memory-bound vs. compute-bound). The variation is not a sign of instability, but rather a demonstration of ARGUS's adaptability. ARGUS addresses these diverse requirements through algorithm-hardware-datatype co-design, ensuring superior performance across all benchmarks. In contrast, existing baselines show uneven performance across tasks because they target a single paradigm: text-based accelerators (FIGNA, AxCore) rely on weight quantization and show moderate gains across all tasks but cannot exploit MLLM-specific redundancies. visual-based accelerators (FlightVGM, S-DMA) achieve gains in generation-heavy tasks (e.g., GEdit-Bench and GenEval) but show limited improvement on Image Understanding (MM-Vet), which is dominated by memory-bound AR decoding.

7.3 Ablation Study

Fig. 12(b) details the cumulative performance contribution of each technique on BAGEL and JanusFlow, reporting Geomean speedup and energy efficiency. Starting from a baseline heterogeneous computing array, incrementally adding T1 (region-structured sparse attention with HSD) yields 1.16 $\times$ speedup and 1.29 $\times$ energy efficiency by eliminating redundant attention computation. Further incorporating T2 (similarity-aware conditional result reuse with SAU) raises performance to 1.24 $\times$ and 1.47 $\times$ by bypassing redundant FFN computation across conditional branches. Finally, enabling T3 (stage-adaptive orchestration) achieves 1.39 $\times$ speedup and 1.65 $\times$ energy efficiency, contributing the largest incremental speedup (1.24 $\times$ $\rightarrow$ 1.39 $\times$) through weight bandwidth compression and workload balancing. These results confirm that all three techniques target complementary bottlenecks and contribute meaningfully to the overall performance.

7.4 Scalability and Energy Efficiency Analysis

Since a single ARGUS tile (2048 MACs, 418.81 mW) operates at a fundamentally different resource budget than the A100 GPU (312 TFLOPS, 380.4 W measured), we evaluate the GPU comparison from two complementary perspectives: scalability under a matched power budget, and energy efficiency at the single-tile level.

Scale-out analysis. We develop an analytical scale-out model to evaluate the latency of a K -tile ARGUS system and compare it against the A100 GPU on GEdit-Bench. The model decomposes single-tile latency into five components and scales each independently: components unaffected by sparsity or reuse (AR-FFN, non-linear operations) scale linearly with K , while Diff-Attn and Diff-FFN scale sub-linearly due to workload imbalance introduced by

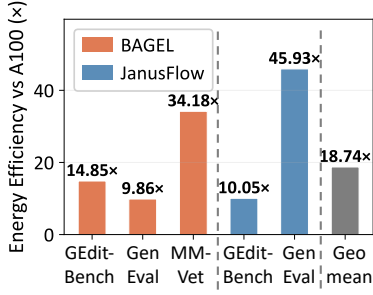


Figure 13: Per-task energy efficiency of a single ARGUS tile relative to the A100 GPU across all benchmarks.

Table 1: Accuracy comparison across BAGEL and JanusFlow.

Models	Methods	GEdit-Bench↑	GenEval↑	MM-Vet↑
BAGEL-7B-MoT	Origin	6.355	0.836	66.8
	S-DMA	4.936 -1.419	0.772 -0.064	66.5 -0.3
	FlightVGM	2.214 -4.141	0.461 -0.375	-
	FIGNA	6.312 -0.043	0.835 -0.001	66.5 -0.3
	AxCore	6.294 -0.061	0.832 -0.004	66.9 $+0.1$
	ARGUS	6.318 -0.037	0.822 -0.014	66.5 -0.3
JanusFlow-1.3B	Origin		0.620	32.3
	S-DMA		0.558 -0.062	32.7 $+0.4$
	FlightVGM	Not Supported	0.305 -0.315	-
	FIGNA		0.620 $+0.000$	34.4 $+2.1$
	AxCore		0.614 -0.006	31.9 -0.4
	ARGUS		0.625 $+0.005$	34.4 $+2.1$

Table 2: Comparison of model accuracy across benchmarks.

Model	BAGEL-7B-MoT			JanusFlow-1.3B	
	GEdit-Bench↑	GenEval↑	MM-Vet↑	GenEval↑	MM-Vet↑
Origin	6.355	0.836	66.8	0.620	32.3
T1-only	6.309 -0.046	-	-	-	-
T2-only	6.333 -0.022	0.822 -0.014	-	0.621 $+0.001$	-
T3-only	6.316 -0.039	0.832 -0.004	66.5 -0.3	0.645 $+0.025$	34.4 $+2.1$
ARGUS	6.318 -0.037	0.822 -0.014	66.5 -0.3	0.625 $+0.005$	34.4 $+2.1$

T1 and T2, and AR-Attn scales near-linearly as T1 also reduces KV cache reads. The imbalance factors for those are calibrated from real attention mask and FFN reuse situation measurements using block tile assignment. Under this model, a $K=94$ -tile ARGUS system matches A100 latency at 39.4 W (10.4% of the A100’s power budget), compared to $K=62$ under ideal linear scaling. This analysis assumes ideal inter-tile bandwidth scaling and does not account for packaging or interconnect overhead.

Per-task energy efficiency. As shown in Fig. 13, the energy efficiency ratio $(T_{\text{GPU}} \times P_{\text{GPU}}) / (T_{\text{ARGUS}} \times P_{\text{ARGUS}})$ reaches a geomean of 18.74 $\times$ across all task-model combinations. AR-dominated tasks (MM-Vet) yield the highest gains (34.18 $\times$ for BAGEL, 45.93 $\times$ for JanusFlow), as single-token decoding is memory-bound and the GPU wastes most of its power on underutilized compute arrays, whereas ARGUS compresses weight bandwidth to match the memory-bound bottleneck. Diffusion-dominated tasks (GenEval) yield lower but still substantial gains ($\sim 10\times$), consistent with the roofline analysis in Section 2.3.

Table 3: Area and power breakdown of ARGUS.

Modules	Configuration	Area (mm^2)		Power (mW)	
HCC	two $4 \times 16 \times 16$ for INT-FP/FP-FP	0.986	59.59%	165.95	39.62%
HSD	16KB ram, $2 \times 256 \times 32$ fifos	0.064	3.85%	28.39	6.78%
SAU	4×16 adder, 4.5KB ram	0.029	1.76%	19.42	4.64%
CRU	16 EW-MUL/ADDs, $2 \times$ LUTs	0.073	4.40%	32.67	7.80%
Buffers	256KB UB, 2×32 KB I/W buffers	0.503	30.40%	172.38	41.16%
Total	-	1.654		418.81	

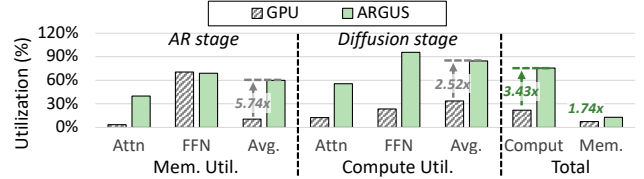


Figure 14: Memory and compute utilization across inference stages, comparing GPU and ARGUS.

7.5 Accuracy Analysis

As shown in Table 1, we compare ARGUS with four representative works and the original implementation of these two MLLMs. Boldface denotes our method, and an underline indicates the best result among the compared methods. For visual-based methods (S-DMA and FlightVGM), we adjust their similarity thresholds to better align with ours in terms of computation and sparsity. As FlightVGM only supports cases where activation in the FFN contains multiple tokens, it cannot be applied in the autoregressive text-generation scenario (MM-Vet). For text-based methods (FIGNA and AxCore), we use their default configurations since they have no adjustable compression hyperparameters. The results show that ARGUS maintains accuracy comparable to the original models across all benchmarks.

Per-technique accuracy impact. As shown in Table 2, each technique individually introduces only marginal accuracy degradation. T1 incurs at most -0.046 on GEdit-Bench. T2 shows at most -0.014 on GenEval, with a slight improvement on JanusFlow ($+0.001$). T3 affects all tasks but remains within -0.3 on MM-Vet. The full ARGUS system achieves accuracy comparable to the per-technique results, confirming that the three techniques compose without compounding accuracy loss.

7.6 Hardware Analysis

7.6.1 Area and Power Breakdown. Table 3 shows the area and power breakdown of ARGUS. The total area is 1.654 mm^2 and total power is 418.81 mW under a 28nm CMOS process. The HCC dominates both area (59.59%) and power (39.62%), as expected for a compute-intensive accelerator. The HSD and SAU, which are the two key novel hardware components, together account for only 5.61% of the total area and 11.42% of total power, demonstrating that the hardware overhead of our co-design approach is modest. This compact power envelope of 418.81 mW per tile directly enables the energy efficiency advantages quantified in Section 7.4, where a single ARGUS tile achieves a geomean of 18.74 $\times$ energy efficiency over the A100 GPU.

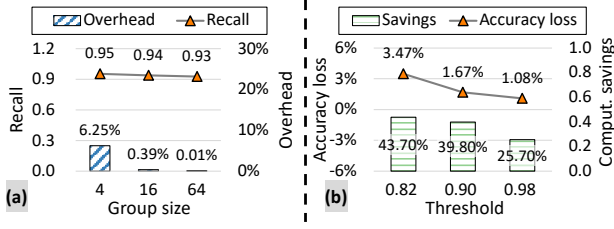


Figure 15: (a) Mask prediction accuracy and overhead under different group sizes. (b) Sensitivity analysis of similarity threshold on computation savings and accuracy.

7.6.2 Utilization Analysis. As shown in Fig. 14, the GPU achieves only 21.95% overall compute utilization, revealing a severe mismatch between the two inference stages. In the AR stage, single-token decoding yields only 10.40% utilization. In the Diffusion stage, conditional branch overhead limits utilization to 33.54%. ARGUS substantially improves the AR stage by compressing FFN weight bandwidth, raising utilization from 10.40% to 59.73% (5.74 $\times$). In the Diffusion stage, eliminating redundant attention and FFN computation raises utilization from 33.54% to 84.51% (2.52 $\times$). Overall, ARGUS achieves 75.36% compute and 12.60% memory bandwidth utilization, a 3.43 $\times$ and 1.74 $\times$ improvement over the GPU baseline (21.95% and 7.25%).

7.7 Sensitivity Analysis

7.7.1 Mask Prediction Accuracy. As shown in Section 2.3, cross-attention and self-attention exhibit fundamentally different sparsity structures, motivating our region-structured sparse attention. This method relies on grouped average pooling to predict attention masks efficiently, and the accuracy of this prediction directly affects the quality of the sparse computation. As shown in Fig. 15(a), we evaluate the mask prediction accuracy by comparing predicted masks against oracle masks across different group sizes G . At $G=4$, the prediction achieves 0.95 recall but incurs 6.25% overhead relative to full attention computation. At $G=64$, the overhead drops to a negligible 0.01% but recall degrades to 0.93. Our choice of $G=16$ strikes the best balance: it achieves 0.94 recall while reducing overhead to 0.39%, providing near-oracle mask quality at little cost.

7.7.2 Similarity Threshold Sensitivity. As shown in Section 2.3, activations across conditional branches exhibit high similarity, motivating our similarity-aware conditional result reuse. This method uses a cosine similarity threshold to determine which tokens can be reused, and the choice of threshold controls the trade-off between computation savings and accuracy. As shown in Fig. 15(b), we sweep the similarity threshold on GenEval and report the resulting computation savings and accuracy change. A lower threshold reuses more aggressively but incurs higher accuracy loss, while a higher threshold is conservative with diminishing savings. We select 0.90 as our default, which achieves 39.80% computation savings with only 1.67% accuracy loss, balancing acceleration and accuracy preservation. The monotonic trend confirms that the similarity patterns arise from the fundamental properties of conditional prediction rather than task-specific characteristics.

8 Related Work

8.1 Text-based Accelerators

Text-based accelerators [6, 14, 27, 29, 30] commonly employ weight quantization to reduce memory access overhead, primarily targeting AR paradigms. For instance, FIGNA [6] proposes an FP-INT multiply-accumulate (MAC) accelerator that relies purely on integer units without numerical approximation. AxCore [30] optimizes errors in numerical approximation and extends Floating Point Multiplication Approximation to support mixed-precision computation. FlightLLM [27] proposes an FPGA-based accelerator that supports flexible mixed-precision computation for LLM inference. Recent efforts have also started to extend the text-based accelerator into multimodal settings. Focus [23] introduces a token-pruning co-design to reduce the significant overhead brought by growing scale and video-level inputs in Vision Language Models (VLMs). V-Rex [7], similarly, proposes a dynamic KV cache retrieval algorithm with a dedicated retrieval accelerator to handle the growing KV cache in streaming VLMs. However, these studies are designed for single-paradigm AR workloads and focus either on quantizing FFN weights or on sparsifying attention, while neglecting their combination within the attention mechanism.

8.2 Visual-based Accelerators

Visual-based accelerators [3, 9, 11, 19, 25, 31] often exploit sparsity or similarity to improve efficiency, primarily targeting diffusion paradigms. S-DMA [31] exploits the similarity inherent in diffusion models to reduce the computational complexity of semantic and spatial sparsity prediction. FlightVGM [11] proposes a spatial-temporal online activation sparsification architecture to reduce redundant computations by exploiting activation similarity in video generation models. Cambricon-D [9] designs a dedicated architecture for diffusion models with optimized dataflow. However, these approaches are designed for single-paradigm diffusion workloads and cannot exploit the region-specific patterns or conditional redundancy inherent to unified MLLMs.

8.3 Summary

The fundamental limitation shared by all existing work is the assumption of a single computational paradigm: text-based accelerators focus on memory-bound AR workloads, while visual-based accelerators target compute-bound diffusion workloads. Works that claim to target multimodal models either address understanding-only tasks (which are essentially AR workloads) or treat generation-only tasks as multimodal, leaving the co-optimization of unified AR+Diffusion MLLMs entirely unaddressed.

9 Conclusion

ARGUS addresses three fundamental challenges (sparsity-accuracy conflict, substantial computational redundancy, and contradictory resource demands) through region-structured sparse attention, similarity-aware conditional result reuse, and stage-adaptive heterogeneous array orchestration. Evaluations across image editing, generation, and understanding tasks demonstrate 1.66 $\times$ ~1.78 $\times$ speedup and 1.43 $\times$ ~18.74 $\times$ energy efficiency improvement over the NVIDIA A100 GPU and state-of-the-art accelerators.

References

- [1] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.
- [2] Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. 2025. *Emerging Properties in Unified Multimodal Pretraining*. <https://doi.org/10.48550/arXiv.2505.14683>
- [3] Li Ding, Jun Liu, Shan Huang, and Guohao Dai. 2025. ViDA: Video Diffusion Transformer Acceleration with Differential Approximation and Adaptive Dataflow. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (Tokyo, Japan) (ASPAC '25)*. Association for Computing Machinery, New York, NY, USA, 148–154. <https://doi.org/10.1145/3658617.3697692>
- [4] Dhruva Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. 2023. GENEVAL: an object-focused framework for evaluating text-to-image alignment. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2270, 21 pages.
- [5] Jonathan Ho and Tim Salimans. 2022. Classifier-Free Diffusion Guidance. *arXiv:2207.12598 [cs.LG]* <https://arxiv.org/abs/2207.12598>
- [6] Jaeyong Jang, Yulhwa Kim, Juheun Lee, and Jae-Joon Kim. 2024. FIGNA: Integer Unit-Based Accelerator Design for FP-INT GEMM Preserving Numerical Accuracy. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 760–773. <https://doi.org/10.1109/HPCA57654.2024.00064>
- [7] Donghyuk Kim, Sejeong Yang, Wonjin Shin, and Joo-Young Kim. 2026. V-Rex: Real-Time Streaming Video LLM Acceleration via Dynamic KV Cache Retrieval. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1–14. <https://doi.org/10.1109/HPCA68181.2026.11408603>
- [8] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.
- [9] Weihao Kong, Yifan Hao, Qi Guo, Yongwei Zhao, Xinkai Song, Xiaqing Li, Mo Zou, Zidong Du, Rui Zhang, Chang Liu, Yuanbo Wen, Pengwei Jin, Xing Hu, Wei Li, Zhiwei Xu, and Tianshi Chen. 2025. Cambricon-D: Full-Network Differential Acceleration for Diffusion Models. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (Buenos Aires, Argentina) (ISCA '24)*. IEEE Press, 903–914. <https://doi.org/10.1109/ISCA59077.2024.00070>
- [10] Yann LeCun. 2022. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review* 62, 1 (2022), 1–62.
- [11] Jun Liu, Shulin Zeng, Li Ding, Widyadewi Soedarmadji, Hao Zhou, Zehao Wang, Jinhao Li, Jintao Li, Yadong Dai, Kairui Wen, Shan He, Yaqi Sun, Yu Wang, and Guohao Dai. 2025. FlightVGM: Efficient Video Generation Model Inference with Online Sparsification and Hybrid Precision on FPGAs. In *FPGA '25: The 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. ACM, 2–13. <https://doi.org/10.1145/3706628.3708864>
- [12] Shiyu Liu, Yucheng Han, Peng Xing, Fukun Yin, Rui Wang, Wei Cheng, Jiaqi Liao, Yingming Wang, Honghao Fu, Chunrui Han, Guopeng Li, Yuang Peng, Qian Sun, Jingwei Wu, Yan Cai, Zheng Ge, Ranchen Ming, Lei Xia, Xianfang Zeng, Yibo Zhu, Binxing Jiao, Xiangyu Zhang, Gang Yu, and Daxin Jiang. 2025. StepIX-Edit: A Practical Framework for General Image Editing. *arXiv:2504.17761 [cs.CV]* <https://arxiv.org/abs/2504.17761>
- [13] Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, Zhiyu Wu, Zizheng Pan, Zhenda Xie, Haowei Zhang, Xing kai yu, Liang Zhao, Yisong Wang, Jiaying Liu, and Chong Ruan. 2025. JanusFlow: Harmonizing Autoregression and Rectified Flow for Unified Multimodal Understanding and Generation. *arXiv:2411.07975 [cs.CV]* <https://arxiv.org/abs/2411.07975>
- [14] Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, and Mao Yang. 2025. LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference. In *ISCA '25: Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ACM, 514–528. <https://doi.org/10.1145/3695053.3731057>
- [15] NVIDIA Corporation. 2024. NVIDIA System Management Interface (nvidia-smi). <https://developer.nvidia.com/nvidia-system-management-interface>. Accessed: 2024-05-20.
- [16] Ritik Raj, Sarbartha Banerjee, Nikhil Chandra, Zishen Wan, Jianming Tong, Ananda Samajdhara, and Tushar Krishna. 2025. SCALE-Sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 186–200.
- [17] Satyabrata Sarangi and Bevan Baas. 2021. DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [18] Weijia Shi, Xiaochuang Han, Chunting Zhou, Weixin Liang, Xi Victoria Lin, Luke Zettlemoyer, and Lili Yu. 2025. LMFusion: Adapting Pretrained Language Models for Multimodal Generation. <https://doi.org/10.48550/arXiv.2412.15188>
- [19] Zhuoran Song, Chunyu Qi, Fangxin Liu, Naifeng Jing, and Xiaoyao Liang. 2024. CMC: Video Transformer Acceleration via CODEC Assisted Matrix Condensing. In *ASPLOS '24: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, 201–215. <https://doi.org/10.1145/3620665.3640393>
- [20] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.
- [21] Inc. Synopsys. [n. d.]. Design Compiler (synopsys.com). <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html> Accessed May 22, 2025.
- [22] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [23] Chiyue Wei, Cong Guo, Junyao Zhang, Haoxuan Shan, Yifan Xu, Ziyue Zhang, Yudong Liu, Qinsi Wang, Changchun Zhou, Hai Helen Li, and Yiran Chen. 2026. Focus: A Streaming Concentration Architecture for Efficient Vision-Language Models. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1–18. <https://doi.org/10.1109/HPCA68181.2026.11408525>
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
- [25] Xujiang Xiang and Fengbin Tu. 2026. VAR-Turbo: Unlocking the Potential of Visual Autoregressive Models Through Dual Redundancy. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1–16. <https://doi.org/10.1109/HPCA68181.2026.11408607>
- [26] Weihao Yu, Zhengyuan Yang, Linjie Li, Jianfeng Wang, Kevin Lin, Zicheng Liu, Xinchao Wang, and Lijuan Wang. 2024. MM-Vet: evaluating large multimodal models for integrated capabilities. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 2381, 25 pages.
- [27] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, Yadong Dai, Jintao Li, Zehao Wang, Ruoyu Zhang, Kairui Wen, Xuefei Ning, and Yu Wang. 2024. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs. In *FPGA '24: The 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. ACM, 223–234. <https://doi.org/10.1145/3626202.3637562>
- [28] Xinjie Zhang, Jintao Guo, Shanshan Zhao, Minghao Fu, Lunhao Duan, Jiakui Hu, Yong Xien Chng, Guo-Hua Wang, Qing-Guo Chen, Zhao Xu, Weihua Luo, and Kaifu Zhang. 2025. Unified Multimodal Understanding and Generation Models: Advances, Challenges, and Opportunities. *arXiv:2505.02567 [cs.CV]*
- [29] Yu Zhang, Mingzi Wang, Lancheng Zou, Wulong Liu, Hui-Ling Zhen, Mingxuan Yuan, and Bei Yu. 2024. MixPE: Quantization and Hardware Co-Design for Efficient LLM Inference. <https://doi.org/10.48550/arXiv.2411.16158>
- [30] Jiaxiang Zou, Yonghao Chen, Xingyu Chen, Chenxi Xu, and Xinyu Chen. 2025. AxCore: A Quantization-Aware Approximate GEMM Unit for LLM Inference. In *MICRO 2025: 58th IEEE/ACM International Symposium on Microarchitecture*. ACM, 839–853. <https://doi.org/10.1145/3725843.3756094>
- [31] Zihan Zou, Xinming Yan, Shun Zhang, Peng Zheng, Guang Yang, Hao Cai, and Bo Liu. 2025. S-DMA: Sparse Diffusion Models Acceleration via Spatiality-Aware Prediction and Dimension-Adaptive Dataflow. In *MICRO 2025: 58th IEEE/ACM International Symposium on Microarchitecture*. ACM, 432–444. <https://doi.org/10.1145/3725843.3756046>